Active Muscle Stress in the MPM Eye

A lecture-note derivation from activation to eye motion

Plexus prototype / Eye G

18 August 2026

What this note derives

A muscle activation in the Eye G model is not converted directly into a force acting on the globe. The activation first creates an *active Cauchy stress* inside the muscle material. The stress is anisotropic and acts along the local muscle-fibre direction. In an MLS-MPM step, the divergence of that stress enters the particle-to-grid momentum transfer. Because the active stress is tapered to zero at the muscle ends, the distributed internal traction is equivalent, at the scale of the whole muscle, to pulling its two endpoints toward one another. If one endpoint is attached to bone and the other is attached to the sclera, the resulting tension shortens the muscle and therefore moves the eye.

The MPM feedback loop in one picture

The easiest coarse description of MPM is a feedback loop between deformation and stress. At time t, each material particle already carries its current deformation gradient F_p^t. The constitutive model turns that deformation into passive elastic stress, while muscle activation contributes an additional active stress. The total stress is then converted into momentum changes on the MPM grid. The updated grid velocity is gathered back to the particles, producing a new local velocity gradient and therefore a new deformation gradient at the next substep. In symbols, the causal loop is

$$F_p^t \longrightarrow \boldsymbol{\sigma}_p^t \longrightarrow \text{grid momentum} \longrightarrow \mathbf{v}_p^{t+\Delta t}, C_p^{t+\Delta t} \longrightarrow F_p^{t+\Delta t}. \tag{1}$$

There is no simultaneous circularity in the calculation: the simulation advances one time step at a time. The deformation already present at time t determines the stress used to compute the motion during the next increment, and that motion changes the deformation carried into the next increment.

For a passive material, deformation creates restoring elastic stress. In an activated muscle, there is also a second entry into the loop: active stress can be non-zero even when $F_p = I$, so activation can initiate contraction. The subsequent deformation then generates passive elastic resistance. In the eye, the observed shortening and rotation are therefore the result of a competition between active contractile stress and passive material, attachment, contact, and surrounding-eye resistance.

1. Start with the biological quantity: activation

For each muscle m, the model carries a scalar activation

$$a_m \in [0, 1]. \tag{2}$$

Activation is a dimensionless control variable. It says how strongly the muscle is being driven, but by itself it is not yet a mechanical force. To turn activation into mechanics, the model multiplies it by a peak active-stress scale A and by spatial factors that describe where along the muscle the force is generated.

For a material point p that belongs to muscle m, let $s_p \in [0, 1]$ denote the fibre coordinate, with $s = 0$ at the proximal origin and $s = 1$ at the distal insertion. The Eye G model uses a taper

$$T(s) = \sqrt{\sin(\pi s)}, \tag{3}$$

so that activation is zero at both ends and maximal in the belly. The point also carries a unit fibre direction $\boldsymbol{f}_p$. In the appendix, $\boldsymbol{f}_p$ is obtained from the gradient of the geodesic fibre coordinate.

The active amplitude at the point is therefore

$$g_p = A\, w_m\, a_m\, T(s_p), \tag{4}$$

where w_m is a dimensionless per-muscle strength weight. Eye G uses $w_m = 1$ for the scanned eye. Here A is the peak active stress (67 in Eye G), not the affine momentum-transfer matrix A_p introduced later.

2. Convert activation into an active stress tensor

A muscle does not pull equally in every spatial direction. It develops tension primarily along its fibres. The simplest tensor that represents this is the outer product

$$\boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}}. \tag{5}$$

This is not a norm and it is not a direction vector. It is a 3×3 tensor constructed from the direction vector. If

$$\boldsymbol{f}_p = \begin{bmatrix} f_x \\ f_y \\ f_z \end{bmatrix}, \qquad \boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}} = \begin{bmatrix} f_x^2 & f_x f_y & f_x f_z \\ f_y f_x & f_y^2 & f_y f_z \\ f_z f_x & f_z f_y & f_z^2 \end{bmatrix}. \tag{6}$$

For normalized $\boldsymbol{f}_p$, this is the projector onto the fibre axis. Applied to any vector $\boldsymbol{d}$,

$$\big(\boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}}\big)\boldsymbol{d} = \boldsymbol{f}_p\big(\boldsymbol{f}_p \cdot \boldsymbol{d}\big). \tag{7}$$

So the active stress

$$\boldsymbol{\sigma}_p^{\mathrm{act}} = g_p\, \boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}} \tag{8}$$

means: **produce tensile stress only along the local fibre axis $\boldsymbol{f}_p$.** Importantly,

$$(-\boldsymbol{f}_p)(-\boldsymbol{f}_p)^{\mathsf{T}} = \boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}}, \tag{9}$$

so fibre orientation has no physical arrowhead: $+\boldsymbol{f}_p$ and $-\boldsymbol{f}_p$ represent the same muscle axis. The tensor is therefore sensitive to the local fibre *axis*, not to an arbitrary choice of arrow direction.

The active stress used in Eye G is

$$\boldsymbol{\sigma}_p^{\mathrm{act}} = A\, w_m\, a_m\, T(s_p)\, \big[1 + \beta(\lambda_p - 1)\big]_+\, \boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}}, \tag{10}$$

Here F_p is the particle's *deformation gradient*, a 3×3 tensor that maps an infinitesimal material vector from the particle's reference configuration into its current deformed configuration. It is the local measure of deformation carried by the MPM particle: stretching, compression, shear, and rotation are all represented in F_p. It is therefore not a stress tensor. The vector $F_p \boldsymbol{f}_p$ is the current image of the reference fibre direction, and its norm gives the current fibre stretch. During the MPM rollout, F_p is updated from the local velocity-gradient state at every substep, so it changes as the eye deforms.

where $\lambda_p = \|F_p \boldsymbol{f}_p\|$ is the current fibre stretch and $[z]_+ = \max(z, 0)$. The term in square brackets is a force–length correction. It compares the current fibre length with its reference length. At initialization $F_p = I$ and $\boldsymbol{f}_p$ is normalized, so $\lambda_p = 1$ and the factor equals one. During an MPM rollout, F_p changes with deformation, so λ_p can change from one time step to the next: $\lambda_p > 1$ means that the material fibre has stretched, while $\lambda_p < 1$ means that it has shortened. If $\beta > 0$, the factor $1 + \beta(\lambda_p - 1)$ therefore increases the active stress when the fibre is stretched and decreases it when the fibre is shortened; the positive-part operator $[\cdot]_+$ prevents the factor from becoming negative. This is a simple linear phenomenological force–length modulation, not a full physiological bell-shaped muscle force–length curve. Crucially, this mechanism is switched off in Eye G: $\beta = 0$. Thus λ_p still evolves because the eye deforms, but it has no effect on the active-stress magnitude in the present Eye G model. In Eye G, the active stress therefore depends on activation, taper and fibre direction, but not on instantaneous fibre stretch.

For Eye G, $\beta = 0$, so Eq. (10) reduces to

$$\boldsymbol{\sigma}_p^{\mathrm{act}} = A\, w_m\, a_m\, T(s_p)\, \boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}}. \tag{11}$$

Thus the active stress has a clear interpretation: its magnitude is set by activation and local taper, and its direction is the muscle fibre direction.

For Eye G, $A = 67$ and the muscle Young's modulus is $E_{\mathrm{muscle}} = 240$, so the dimensionless active-stress scale is

$$\frac{A}{E_{\mathrm{muscle}}} = \frac{67}{240} \approx 0.28. \tag{12}$$

This ratio is useful because it compares the maximum active stress with the passive stiffness scale of the same muscle.

Eye G: active stress versus passive stiffness

The two numbers $A = 67$ and $E_{\mathrm{muscle}} = 240$ should be read as two different mechanical scales. A is the maximum local active stress generated by a fully activated muscle particle at the centre of the muscle when $w_m = 1$, $T(s) = 1$, and the force–length factor is one. It is therefore the amplitude of the contractile stress source. $E_{\mathrm{muscle}} = 240$ is the Young's modulus of the passive muscle material. It sets the scale of the elastic stress that appears when the material is deformed away from its

reference configuration. The ratio

$$\chi_{\text{active/passive}} = \frac{A}{E_{\text{muscle}}} = \frac{67}{240} \approx 0.279, \tag{13}$$

is a useful dimensionless measure of how large the available active stress is compared with the muscle's passive elastic stiffness scale.

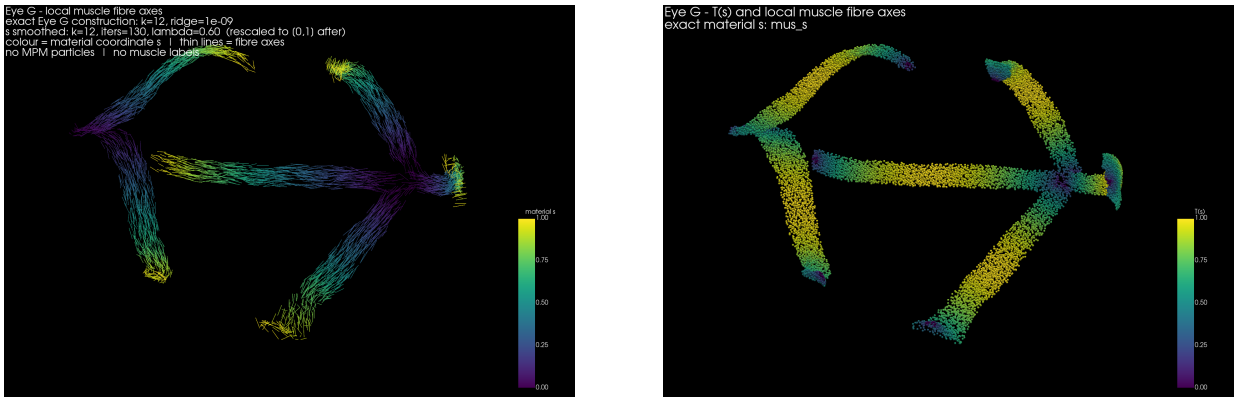

Figure 1: Left: the fibre direction field $\boldsymbol{f}_p$ on the six muscle straps. Right: the active-stress taper $T(s_p)$ along the fibre coordinate, zero at the bony origin and the scleral insertion, peaking over the belly.

This ratio is *not* a prediction that the muscle shortens by 28%. Young's modulus and active stress have the same units, but the resulting strain also depends on the geometry of the muscle, its cross-sectional area, the fibre taper, its attachments, the stiffness of the surrounding eye, contact, and the full nonlinear MPM dynamics. A useful first intuition is nevertheless that, for a fixed active stress, increasing E_{muscle} makes a given deformation harder to produce, whereas decreasing E_{muscle} makes the same active stress more effective at deforming the muscle. In other words, A is the strength of the internal contraction and E is the resistance offered by the material itself.

For Eye G the passive stiffness varies strongly across the eye, so the muscle does not contract against a single uniform elastic background. The scanned model uses $E = 240$ for the muscle straps, while the globe is heterogeneous: $E = 45$ for the soft core/vitreous region, $E = 130$ at intermediate depth, and $E = 420$ in the sclera. The cornea is $E = 320$ and the lens is $E = 520$. The model also contains explicit attachment and surrounding-tissue stiffnesses: the orbit/socket uses approximately $k = 5000$, orbital fat $k = 4000$, and the tendon/bone anchoring $k = 9000$. These are different kinds of parameters from Young's modulus, so they should not be compared by taking their numerical ratios directly; E is a material stress–strain scale, whereas k is a penalty/attachment stiffness in the force law.

The Eye G values also show why "more active stress" does not simply mean "more useful eye motion." The characterised muscle amplitude is $A = 67$. Increasing the drive by $1.5\times$ moves the active stress scale toward 100.5, but the observed response is not a $1.5\times$ increase in useful eye rotation. In the Eye G tests, that increase causes substantial nonlinear deformation: the globe loses about 6.4% of its radius, peak muscle shortening rises from about 26% to about 74%, and the simulation enters a regime with numerical failure indicators. This is the practical consequence of the competition represented by

$$\sigma_p = \sigma_p^{\text{el}} + \sigma_p^{\text{act}}. \tag{14}$$

At moderate activation, active stress produces contraction while passive elasticity and the surrounding eye return forces oppose deformation. At excessive activation, the active term can drive the material into large strains, contact changes and numerical instability before proportional anatomical motion is obtained.

Thus the most useful interpretation of the Eye G numbers is not "67 versus 240", but the hierarchy they create:

$$\text{active stress scale} \quad \text{versus} \quad \text{muscle elastic scale} \quad \text{versus} \quad \text{surrounding-eye and attachment constraints.} \tag{15}$$

The eye rotation that we ultimately observe is the result of all three acting together.

Material and mechanical levers that change Eye G dynamics

Coarse view: what makes the eye turn farther, and what makes it move faster? A useful first question is not "which parameter is the muscle strength?" but rather: *which parameter changes the achievable gaze, and which parameter changes the time course by which the eye reaches it?* At coarse scale, the gaze excursion is controlled by the competition between active contractile stress, mechanical leverage, and passive resistance:

$$\boxed{\text{gaze amplitude} \sim \frac{\text{active muscle drive} \times \text{mechanical leverage}}{\text{passive mechanical resistance}}}. \tag{16}$$

For Eye G, increasing the peak active stress A generally increases the available contractile drive and tends to increase excursion; increasing muscle or surrounding-eye stiffness tends to reduce the deformation produced by that drive. Changing muscle length, fibre path or insertion position changes the mechanical leverage, so the same stress can produce a different torque about the eye centre. Mass and inertia are different: they mainly influence how rapidly the eye accelerates and decelerates rather than its ideal static equilibrium, although coupled nonlinear dynamics can make the separation imperfect.

The corresponding coarse view of the transient response is

$$\boxed{\text{gaze dynamics} \sim \text{active drive, inertia, stiffness, damping, and endpoint constraints}}. \tag{17}$$

Thus there are two useful classes of lever. Parameters such as A, passive stiffnesses, endpoint geometry and insertion geometry strongly affect how far the eye can be driven. Parameters such as mass, rotational inertia, damping and stiffness also affect how fast it gets there, how much it overshoots, and how much lag is produced. The present Eye G force–length coefficient is a special case: $\beta = 0$, so instantaneous fibre stretch does not currently modulate the active-stress magnitude.

Exhaustive inventory of the mechanical levers The following inventory expands that coarse picture into the individual parameters used by the Eye G model.

Peak active stress A. $A = 67$ sets the amplitude of the active contractile stress at a fully activated muscle particle in the belly when $w_m = 1$, $T(s) = 1$, and the force–length factor is one. Increasing A increases the stress available to shorten the muscle and therefore generally increases the torque transmitted to the eye, until contact, excessive deformation or numerical limits intervene.

Muscle Young's modulus E_{muscle}. Eye G uses $E_{\text{muscle}} = 240$. It sets the passive elastic stress scale of the muscle through the Lam'e parameters. Increasing E_{muscle} makes a given active stress harder to convert into deformation; decreasing it makes the muscle more compliant. The dimensionless comparison $A/E_{\text{muscle}} = 67/240 \approx 0.28$ is useful, but it is not a prediction of 28% shortening because geometry and loading determine the actual deformation.

Force–length coefficient β. The factor $[1 + \beta(\lambda_p - 1)]_+$ can modulate active stress as the fibre stretches or shortens. In the present Eye G model $\beta = 0$, so this lever is intentionally switched off. If enabled, it would provide an additional dynamic feedback from fibre deformation into active stress.

Muscle length, cross-section, volume and fibre path. These geometric quantities determine where the active stress is distributed, how much material is accelerated, how the muscle deforms, and what line of action and moment arm it presents to the eye. A change in muscle length therefore affects more than one mechanism at once: mass, stiffness distribution, curvature, and endpoint geometry can all change.

Insertion geometry and endpoint constraints. The proximal origin determines where the reaction is supplied by bone; the distal insertion determines where the muscle traction enters the sclera. Their positions and orientations set the torque arm. The endpoint stiffnesses determine how much the attachments themselves yield rather than transmitting load to the eye. Consequently, two muscles with the same A, E and activation can produce different eye rotations.

Muscle mass and density. Eye G uses density $\rho = 1$ for the scanned soft tissues and assigns each particle $m_p = \rho V_p^0$. Because the six muscles have different anatomical volumes, they have different total masses despite the same density. With a fixed particle count, they also have different particle reference volumes and therefore different particle masses. Increasing muscle mass mainly increases inertia and therefore slows the response to a given active stress; it does not directly increase the stress generated at a fixed activation.

Eye mass and rotational inertia. The globe and its attached tissues have inertia. More mass, or a larger effective moment of inertia about the relevant rotation axis, means that a given muscle torque produces less angular acceleration. These parameters therefore primarily affect transient timing, overshoot and lag rather than the static equilibrium when the same stiffness and active stress are retained.

Eye tissue Young's moduli. The eye is heterogeneous. Eye G uses $E = 45$ for the soft core/vitreous region, $E = 130$ at intermediate depth, $E = 420$ for sclera, $E = 320$ for cornea and $E = 520$ for lens, while the muscle straps use $E = 240$. These elastic moduli control how much each tissue deforms under load and therefore how much of a muscle contraction appears as globe rotation rather than local deformation.

Socket, orbital fat, and bone/tendon stiffness. Eye G includes explicit stiffnesses of approximately $k = 5000$ for the socket/orbit, $k = 4000$ for orbital fat, and $k = 9000$ for proximal tendon/bone anchoring. These k values are not Young's moduli and should not be compared numerically with E: they belong to explicit penalty or attachment force laws. Increasing them generally makes the corresponding constraint yield less and can transmit more of the muscle load into globe motion, while decreasing them allows more local compliance.

Transverse sleeve and path constraints. The optional sleeve penalizes displacement and velocity perpendicular to the local fibre while leaving axial shortening relatively free. It therefore

changes the route by which the muscle deforms - especially buckling and lateral displacement - without directly prescribing how much it shortens. It can strongly change the efficiency with which active contraction produces useful eye rotation.

Damping. Damping does not primarily determine the static gaze produced by a held activation, but it changes how the eye approaches that state: it controls the amount and duration of oscillation and therefore affects overshoot, settling time and apparent lag.

The complete coarse-to-detailed hierarchy can therefore be summarized as

$$\boxed{\text{active stress } \rightarrow \text{ muscle deformation } \rightarrow \text{ endpoint loading } \rightarrow \text{ eye deformation and rotation}}, \tag{18}$$

while mass, rotational inertia and damping mainly control how rapidly the response unfolds, and passive stiffnesses, geometry and endpoint constraints control how strongly motion is resisted and where the system settles. Numerical parameters such as grid spacing, MPM substep size and numerical damping are separate from these biological and material levers: they can change computed stability or temporal resolution without representing a change in the modeled Eye G material itself.

3. Add active stress to the passive elastic stress

The total stress carried by the muscle particles is the sum

$$\boldsymbol{\sigma}_p = \boldsymbol{\sigma}_p^{\mathrm{el}} + \boldsymbol{\sigma}_p^{\mathrm{act}}. \tag{19}$$

The passive part in the model is fixed-corotated elasticity. In the appendix it is written

$$\boldsymbol{\sigma}_p^{\mathrm{el}} = 2\mu_p(F_p - R_p)F_p^{\mathsf{T}} + \lambda_p^{\mathrm{L}} J_p(J_p - 1)I, \tag{20}$$

with R_p the polar rotation, $J_p = \det F_p$, and Lamé parameters determined from Young's modulus and Poisson ratio.

The distinction is important. Passive elasticity resists deformation; active stress is the internal contractile action that drives deformation. The observed shortening is the result of their competition.

4. How stress becomes a force in MPM

In the MLS-MPM discretisation, internal forces are not applied as separate point forces. Instead, each material point transfers momentum to the background grid. For a grid node i and particle p, the affine particle-to-grid transfer is

$$(m\boldsymbol{v})_i \mathrel{+}= w_{ip}\left[m_p\boldsymbol{v}_p + A_p(\boldsymbol{x}_i - \boldsymbol{x}_p)\right], \tag{21}$$

Here m_p is the mass represented by material particle p. In Eye G it is defined from the particle reference volume and the assigned material density,

$$m_p = \rho V_p^0. \tag{22}$$

For the scanned soft tissues Eye G uses the same density $\rho = 1$, while preserving the measured differences in anatomical volume. Thus muscles with different scanned volumes have different total

masses even though their material density is identical; with a fixed particle count, their per-particle reference volumes and therefore their per-particle masses also differ. The m_p in Eq. (21) is therefore not a unit weight inserted for convenience: it carries the inertia represented by the volume assigned to particle p. with

$$A_p = m_p C_p - \frac{4\,\Delta t_{\text{sub}}}{\Delta x^2} V_p^0 \boldsymbol{\sigma}_p. \tag{23}$$

Here w_{ip} is the **particle-to-grid interpolation weight**, also called the MPM **shape-function weight**. It is the value of the quadratic B-spline basis connecting particle p to grid node i: nearby nodes receive a large fraction of the particle contribution and nodes outside the local support receive zero. In three dimensions the quadratic basis has support on three nodes per coordinate, so one particle communicates with a local $3 \times 3 \times 3 = 27$-node neighbourhood. The weight is numerical rather than material: it does not represent a force or stiffness; it specifies how particle information is distributed onto the temporary grid.

The matrix A_p is the **APIC/MLS-MPM particle affine momentum tensor**. It is not itself the stress tensor. It packages two effects that both vary linearly with the node offset $(\boldsymbol{x}_i - \boldsymbol{x}_p)$:

$$A_p = \underbrace{m_p C_p}_{\text{affine velocity contribution}} - \underbrace{\frac{4\,\Delta t_{\text{sub}}}{\Delta x^2} V_p^0 \boldsymbol{\sigma}_p}_{\text{stress contribution}}. \tag{24}$$

Here C_p is the particle's **local affine velocity-gradient matrix**. It is a 3×3 matrix that describes how the velocity varies in a small neighbourhood of particle p. Locally, the APIC representation may be read as

$$\mathbf{v}(\mathbf{x}) \approx \mathbf{v}_p + C_p(\mathbf{x} - \mathbf{x}_p), \tag{25}$$

so, in the continuum interpretation,

$$C_p \approx \nabla \mathbf{v}\big|_p = \begin{bmatrix} \partial v_x/\partial x & \partial v_x/\partial y & \partial v_x/\partial z \\ \partial v_y/\partial x & \partial v_y/\partial y & \partial v_y/\partial z \\ \partial v_z/\partial x & \partial v_z/\partial y & \partial v_z/\partial z \end{bmatrix}_p. \tag{26}$$

The diagonal entries describe local extension or compression rates along the coordinate axes, while off-diagonal entries contribute to local shear and rotation. In APIC/MLS-MPM, C_p is reconstructed from the grid-to-particle velocity field, so calling it exactly $\nabla \mathbf{v}$ is an approximation: it is the best local affine representation carried by the particle. Its units are inverse time. By contrast, F_p is the accumulated deformation gradient: C_p describes how the material is moving now, while F_p records how it has deformed over time. The deformation update used by the MPM integrator is $F_p^{t+\Delta t} = (I + \Delta t\, C_p) F_p^t$. Here V_p^0 is the particle's reference volume, Δx is the **three-dimensional MPM grid spacing** (the distance between neighboring grid nodes), and Δt_{sub} is the MPM substep. The appearance of Δx^2 below does **not** mean that the simulation is two-dimensional: the square comes from the spatial derivative and quadratic B-spline MLS-MPM discretization of the stress divergence. Eye G is fully three-dimensional; its grid is 112^3, giving $\Delta x \approx 8.9 \times 10^{-3}$ for the unit box. Multiplication by the offset converts this particle-level matrix into the contribution appropriate to a particular grid node:

$$\begin{aligned} A_p(\boldsymbol{x}_i - \boldsymbol{x}_p) = {} & m_p C_p(\boldsymbol{x}_i - \boldsymbol{x}_p) \\ & - \frac{4\,\Delta t_{\text{sub}}}{\Delta x^2} V_p^0 \boldsymbol{\sigma}_p(\boldsymbol{x}_i - \boldsymbol{x}_p). \end{aligned} \tag{27}$$

The first term preserves the local affine variation of velocity around the particle; the second converts internal stress into a spatially antisymmetric momentum transfer. Equation (21) then multiplies the complete contribution by w_{ip} before adding it to node i.

The active contribution is obtained by inserting only $\boldsymbol{\sigma}_p^{\mathrm{act}}$ into Eq. (23):

$$A_p^{\mathrm{act}} = -\frac{4\,\Delta t_{\mathrm{sub}}}{\Delta x^2} V_p^0 \boldsymbol{\sigma}_p^{\mathrm{act}}. \tag{28}$$

Using Eq. (11),

$$A_p^{\mathrm{act}} = -\frac{4\,\Delta t_{\mathrm{sub}} V_p^0}{\Delta x^2}\, g_p\, \boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}}. \tag{29}$$

The active impulse deposited on node i is therefore proportional to

$$A_p^{\mathrm{act}}(\boldsymbol{x}_i - \boldsymbol{x}_p) = -\frac{4\,\Delta t_{\mathrm{sub}} V_p^0 g_p}{\Delta x^2}\, \boldsymbol{f}_p\big(\boldsymbol{f}_p \cdot (\boldsymbol{x}_i - \boldsymbol{x}_p)\big). \tag{30}$$

Equation
eqrefeq:rank-one-force is the active-muscle specialization of the stress part of A_p. Because
$sig_p^{rmact} = g_p$
$vectf_p$
$vectf_p^{mathsfT}$ is rank one, applying it to the node offset first projects that offset onto the fibre axis and then returns a vector along the same axis. Thus the active stress cannot directly push sideways relative to the local fibre: its node-level momentum contribution is axial.

This is the key mechanical step. The scalar product

$$\boldsymbol{f}_p \cdot (\boldsymbol{x}_i - \boldsymbol{x}_p) \tag{31}$$

measures how far the grid node lies ahead of or behind the particle along the fibre. The resulting vector is parallel to $\boldsymbol{f}_p$, and its sign reverses on opposite sides of the particle. Nodes ahead of the particle are therefore pulled back toward it, while nodes behind are pulled forward. The stress field consequently produces a local inward pull along the fibre.

5. From distributed stress to endpoint tension

Consider a straight muscle segment with fibre coordinate s and cross-sectional area S. In a one-dimensional approximation, the axial active stress is

$$\sigma_{\parallel}(s) = g(s). \tag{32}$$

The corresponding axial force is

$$F(s) = S\,\sigma_{\parallel}(s). \tag{33}$$

Mechanical equilibrium says that the distributed internal force density is the spatial derivative of this axial force,

$$f_{\mathrm{body}}(s) = \frac{\mathrm{d}F}{\mathrm{d}s}, \tag{34}$$

up to the sign convention used for stress divergence.

Because the Eye G taper satisfies

$$T(0) = T(1) = 0, \tag{35}$$

the active stress rises from zero near the origin, reaches a maximum in the belly, and falls back to zero near the insertion. This taper is important: the active term is a distributed stress field rather than a point force applied at an endpoint. Its divergence produces internal forces throughout the strap, with opposite signs on the two sides of each material point. The local result is always an inward pull along the fibre.

The endpoint interpretation appears when that distributed stress field is connected to anatomy. In a continuum description, the traction acting across any internal cut with normal $\boldsymbol{n}$ is

$$\boldsymbol{t} = \boldsymbol{\sigma}\boldsymbol{n}. \tag{36}$$

For a cut normal to the local fibre, $\boldsymbol{n} = \pm\boldsymbol{f}$, the axial component is approximately

$$\boldsymbol{t} = \pm\sigma_{\parallel}\boldsymbol{f}. \tag{37}$$

Thus an imaginary cut through the muscle has two equal-and-opposite axial tractions. Those tractions are transmitted through the elastic muscle toward the proximal and distal attachments. The physical endpoint caps then determine where that contraction can go: the proximal attachment is held by bone, while the embedded distal attachment transmits the resulting tension to the globe. In this sense, the muscle behaves at the anatomical scale like a tension element whose active stress tries to reduce the distance between its attachments, even though no point muscle force" is prescribed in the MPM equations.

6. Why the endpoints determine whether the eye moves

The two ends of a muscle do not play the same geometric role. In the Eye G model, the proximal end is attached to bone, while the distal end is embedded at the sclera. The proximal attachment supplies a reference position; the distal attachment transfers the muscle's contraction to the globe.

Proximal endpoint. The bone-side cap is anchored to a fixed anatomical origin. In the penalty formulation, particles in the proximal region experience

$$\boldsymbol{a} = k(\boldsymbol{x}^{\text{rest}} - \boldsymbol{x}) - c\boldsymbol{v}, \tag{38}$$

which pulls them back toward their rest location and damps their motion. The alternative pinning implementation makes this attachment kinematic: the selected region is reset to its rest position each substep.

Distal endpoint. The insertion is treated differently. The muscle belly is kept a small clearance away from the globe so that the shared MPM grid does not weld the whole muscle to the sclera. Only the distal cap is embedded. Thus the active tension generated throughout the strap is transmitted to the globe mainly through a localized insertion rather than through accidental contact along the muscle length.

The shared grid is what lets this work mechanically. Globe and muscles occupy different particle sets, but their particles communicate through the same grid. Momentum deposited by the muscle is transferred to the common grid and then back to the globe where the insertion provides mechanical

coupling. The bone-side attachment prevents the whole muscle from simply translating. The result is therefore a relative contraction: the muscle deforms between a comparatively fixed origin and a moving insertion, and that insertion transmits the load to the globe. The eye moves because the active stress has changed the momentum balance of the coupled mechanical system.

6a. From material points to interacting biological bodies

An MPM particle should not be pictured as an independent bead in a cloud. It is a moving sample of continuum material: it carries position and velocity, but also mass, reference volume, deformation gradient F_p, material parameters and, in the eye, biological information such as muscle identity and fibre direction. A Plexus entity such as the globe or the lateral rectus is therefore represented mechanically by many such material samples. The particles do not need permanent springs connecting them. Instead, their collective identity as an elastic body is recovered at every MPM substep by the cycle particle $\rightarrow$ grid $\rightarrow$ particle. During P2G, neighboring particles deposit mass, momentum and stress contributions onto common grid nodes. The grid therefore reconstructs a local continuum momentum field from the discrete samples. After forces have changed this field, G2P returns the resulting motion to the particles and updates their deformation gradients. Deformation is remembered by F_p, and the constitutive law converts that deformation back into elastic stress on the following step. The repeated loop

$$\text{motion} \rightarrow F_p \rightarrow \text{stress} \rightarrow \text{grid momentum} \rightarrow \text{motion} \tag{39}$$

is what makes thousands of moving material points behave as a deformable globe or muscle rather than as an unstructured particle cloud.

This also explains how one Plexus entity can mechanically move another without either losing its identity. The globe and each muscle are separate entities and their particles retain different material parameters, fibre fields and biological roles, but they scatter onto the same MPM grid. When an activated muscle develops the rank-one stress $\boldsymbol{\sigma}^{\text{act}} = g\,\boldsymbol{f}\boldsymbol{f}^{\mathsf{T}}$, that stress changes momentum on the grid nodes supporting the muscle. At the insertion, scleral particles have overlapping grid support and therefore participate in the same local momentum update. When velocities are gathered back from the grid, the scleral particles acquire motion generated by the contracting muscle. Thus the mechanical causal chain is

$$\text{muscle activation} \rightarrow \text{active stress} \rightarrow \text{shared grid momentum} \rightarrow \text{scleral motion} \rightarrow \text{globe rotation}. \tag{40}$$

No explicit force is applied from a `muscle'' object to an` eye" object. The interaction emerges one level below the entities, from momentum exchange between their material discretizations.

The endpoints determine what that internally generated contraction can accomplish. At the proximal end, the muscle is constrained relative to bone; at the distal end, its insertion is mechanically coupled to the sclera. Active stress therefore cannot simply translate the whole strap through space. Shortening between the two constrained regions instead loads the attachments, so the distal attachment pulls on the globe while the proximal attachment supplies the reaction. Because the insertion lies away from the globe's centre, this traction generates a torque and rotates the eye. Importantly, the active-stress taper makes the active term vanish at the mathematical ends of the fibre: the attachment force is not an artificial endpoint force inserted by hand, but stress transmitted through the passive elastic material from the active muscle belly into its attachments.

Contact requires a further distinction. A shared-grid MPM calculation strongly couples materials whose particles contribute to the same grid nodes, but this should not be interpreted as a universal guarantee that different bodies can never interpenetrate. In the present Eye G formulation, excessive overlap between muscle and globe behaves approximately as a weld: the two populations exchange momentum so strongly that relative sliding is suppressed. This is why the scanned muscle belly is seeded with a finite standoff from the globe while only the distal cap is embedded into the sclera. The same numerical mechanism is useful at the insertion, where attachment is wanted, and undesirable along the belly, where biological sliding is wanted. A more general contact formulation would explicitly distinguish nonpenetration, separation, sticking and frictional sliding rather than obtaining these behaviours primarily from geometry and shared-grid coupling.

Finally, deformation, fracture and collision should not be conflated. Elasticity allows an entity to change shape while its deformation gradient continues to transmit restoring stress through the material. Tearing requires an additional damage or fracture rule that weakens or removes that stress transmission once a failure criterion is reached. After separation, the resulting pieces may still interact through a contact law. MPM therefore provides the bottom-up mechanical substrate, while distinct constitutive, damage and contact mechanisms determine whether material locally behaves as one deformable body, separates into pieces, or collides with another body. In Plexus terms, entity identity describes *what biological object a material sample belongs to*, whereas the MPM operators determine *how those objects exchange momentum and deform in space.*

7. How stiffness and elasticity control shortening

Active stress does not prescribe a fixed shortening distance. Shortening emerges from the balance between the active stress and the passive elastic resistance. A useful first scaling is

$$\varepsilon_{\mathrm{act}} \sim \frac{\sigma_{\mathrm{act}}}{E}, \tag{41}$$

where $\varepsilon_{\mathrm{act}}$ is a characteristic strain and E is Young's modulus. If the active stress is held fixed and E is increased, the same muscle develops less strain: the muscle becomes harder to shorten. If E is decreased, the same active stress produces more deformation. In a soft-body eye, however, excessive softness is not automatically better: the strap can buckle, over-stretch, or deform in directions that do not contribute usefully to rotation.

Elasticity also determines how the muscle stores and returns mechanical energy. During activation, part of the active work is spent deforming the passive tissue. When activation changes, that stored elastic energy contributes to the transient response. Consequently, the active-to-passive ratio A/E controls not only how much a muscle can shorten but also how sharply the system responds. In Eye G, $A/E_{\mathrm{muscle}} \approx 0.28$ is already large enough to produce substantial shortening, while increasing the drive beyond the characterised regime causes severe deformation and numerical problems rather than simply producing a proportionally larger eye rotation. The eye is therefore constrained jointly by anatomy, active stress, passive stiffness, and numerical resolution.

8. The complete causal chain

The whole mechanism can now be read as a sequence:

activation → active stress → stress divergence
→ grid momentum transfer → attachment loading
→ muscle shortening → scleral force → eye rotation

No stage is a prescribed move the eye" command. The model specifies local activation, fibre geometry, material stress, attachment, and MPM coupling. The eye motion is the resulting solution of the mechanical system.

Interpretation for Plexus

The important modelling abstraction is therefore not `muscle force'' but` active stress emitted on a material field." This keeps contraction local, anisotropic, geometry-aware, and compatible with the same MPM machinery used for passive elasticity. Endpoint forces and eye motion are emergent consequences of stress transmission and attachment. That is exactly the kind of mechanism that a Plexus operator should expose without hard-coding the final anatomical outcome.

A Appendix: the MPM implementation, equation by equation

This appendix connects the continuum equations in the lecture note to the actual Plexus implementation used by the Eye G prototype. The goal is not to reproduce complete source files, but to show the decisive lines for each mathematical operation. The listings use the current public `main` branch. The file and repository location are given in each caption so that the code can be inspected in context. The code is shown in small script font with source line numbers corresponding to the current files.

A useful way to read the implementation is as a four-stage MPM cycle:

$$\text{material update} \rightarrow \text{particle-to-grid scatter} \rightarrow \text{grid solve} \rightarrow \text{grid-to-particle gather}. \tag{42}$$

The muscle-specific operators sit on top of this cycle: `muscle_morphogenesis` establishes the material geometry, `muscle_geometry` measures the current body, `muscle_contract` writes active stress, and `bone_anchor` constrains the proximal cap.

A.1 Where the particle fibre direction $\boldsymbol{f}_p$ is first computed

For the scanned Eye G eye, the particle-wise fibre direction is not guessed by `muscle_contract`. It is created during the `blend_muscles` Seed operator in `prototype/eye/blend_mpm_ops.py`. The sequence is important. First, `path_coordinate` assigns every sampled muscle particle a material coordinate s_p by computing a shortest-path distance through the particle cloud from the insertion and flipping it so that $s = 0$ is the proximal origin and $s = 1$ is the distal insertion. Then `fibre_from_s` computes a local gradient of that scalar field. The resulting normalized gradient is the actual particle-wise fibre vector $\boldsymbol{f}_p$ used later by `muscle_contract`. The public implementation explicitly describes this as a local gradient construction: the least-squares relation $(\boldsymbol{x}_j - \boldsymbol{x}_i)\cdot\boldsymbol{g}_i = s_j - s_i$ is solved over each particle's neighbourhood, and the normalized solution is stored as the fibre direction.

Thus the construction is bottom-up:

$$\text{scanned muscle surface} \longrightarrow \{\boldsymbol{x}_p\} \longrightarrow \{s_p\} \longrightarrow \{\nabla s_p\} \longrightarrow \{\boldsymbol{f}_p\}. \tag{43}$$

This is particularly important for bent muscles. A single global tangent would fail when the strap wraps around the globe; the gradient of the geodesic coordinate changes locally, so each particle obtains its own fibre axis. The implementation itself records that this replaced a straight-axis construction that produced an empty centreline bin for SR and a grossly incorrect length.

Listing 1: `prototype/eye/blend_mpm_ops.py`: geodesic material coordinate s_p and the local fibre direction $\boldsymbol{f}_p$. Source lines 2037–2112 on the current `main` branch.

```
def path_coordinate(pts, seed_pt, k=12):
    tree = cKDTree(pts)
    dist, idx = tree.query(pts, k=k + 1)
    rows = np.repeat(np.arange(len(pts)), k)
    G = csr_matrix((dist[:, 1:].ravel(),
                    (rows, idx[:, 1:].ravel())),
                   shape=(len(pts), len(pts)))
    G = G.maximum(G.T)
    src = np.argsort(((pts - np.asarray(seed_pt)[None, :]) ** 2).sum(1))[:max(8, len(pts) // 200)]
    d = dijkstra(G, directed=False, indices=src, min_only=True)
    d[~np.isfinite(d)] = d[np.isfinite(d)].max()
    return 1.0 - d / max(d.max(), 1e-12), idx

def fibre_from_s(pts, s, idx, ridge=1e-9):
    dx = pts[idx[:, 1:]] - pts[:, None, :]
    ds = s[idx[:, 1:]] - s[:, None]
    A = np.einsum('nki,nkj->nij', dx, dx) + ridge * np.eye(3)[None]
    b = np.einsum('nki,nk->ni', dx, ds)
    g = np.linalg.solve(A, b[..., None])[..., 0]
    return g / np.linalg.norm(g, axis=1, keepdims=True).clip(1e-12)
```

The key calculation is the least-squares gradient. For one particle i and its neighbours j, we seek a vector $\boldsymbol{g}_i$ satisfying

$$(\boldsymbol{x}_j - \boldsymbol{x}_i) \cdot \boldsymbol{g}_i \approx s_j - s_i. \tag{44}$$

If the scalar field s increases along the muscle, then $\boldsymbol{g}_i$ points toward increasing s, namely from origin toward insertion. The code solves the normal equations for this local gradient and then normalizes it:

$$\boldsymbol{f}_p = \frac{\nabla s_p}{\|\nabla s_p\|}. \tag{45}$$

Because the calculation uses the local neighbourhood rather than a single centreline segment, the direction can rotate continuously as the muscle bends. This is the quantity that later enters $\boldsymbol{f}_p \boldsymbol{f}_p^{\mathsf{T}}$.

The actual call that creates the field and the persistent particle buffer occurs inside `BlendMuscles.forward`. The material coordinate is recomputed after the optional standoff/embedding correction, and then the normalized local fibre direction is written into the particle state.

Listing 2: `prototype/eye/blend_mpm_ops.py`: creation and storage of the particle fibre field. Source lines 2624–2698.

```
pts, vol = fill_meshes([(V, F)], sel.size, device=dev)
ins = fr(d[f"{part}__centreline__v"])[0]
s, idx = path_coordinate(pts, ins)

if self.standoff != 0.0 or self.embed != 0.0:
    pts, moved = relieve_overlap(
        pts, s, self.center, shell,
```

```
        self.standoff, self.embed,
        self.cap, device=dev,
    )
    s, idx = path_coordinate(pts, ins)

X[sel] = pts
fib[sel] = fibre_from_s(pts, s, idx)
sarr[sel] = s

p.register_buffer(
    "fibre",
    torch.as_tensor(fib, dtype=torch.float32, device=dev),
)
p.register_buffer(
    "s",
    torch.as_tensor(sarr, dtype=torch.float32, device=dev),
)
```

The final storage line is the bridge to the dynamics. Once the Seed has run, every muscle particle carries its own s_p and $\boldsymbol{f}_p$. The later contractile operator does not reconstruct the fibre direction; it simply reads the stored field:

Listing 3: `prototype/eye/muscle_ops.py`: the dynamics operator reads the stored particle fibre field. Source lines 2371–2388.

```
p = H.level(self.at)
m = H.level(self.muscles)

if not hasattr(p, "fibre"):
    return {}

act = m.get("act")[:, 0].clamp(min=0.0)[p.parent]
f = p.fibre
gate = self.amplitude * self._strength_t[p.parent] * act
if self.taper:
    gate = gate * torch.sin(math.pi * p.s.clamp(0, 1)).clamp(min=0.0) ** 0.5
```

This gives the precise answer to "where does $\boldsymbol{f}_p$ first exist?" for the scanned Eye G implementation: it is first computed by `fibre_from_s` inside the `blend_muscles` Seed, then stored as the `fibre` buffer on the `muscle_particle` set. `muscle_contract` consumes that same particle-wise field and forms the active-stress tensor from it. The repository documentation describes `blend_muscles` as the Seed that gives muscle particles their fibre coordinate and local fibre direction, while `muscle_contract` is the later exchange operator that uses the field to generate active stress.[1, 2]

A.2 Active-stress amplitude and fibre tensor

The equation

$$g_p = A\, w_m\, a_m\, T(s_p) \tag{46}$$

is implemented by the `muscle_contract` operator. Activation is first broadcast from the parent muscle to its material points through the `parent` map; the taper then removes active stress from the tendon regions.

Listing 4: `prototype/eye/muscle_ops.py`: activation, amplitude, and tendon taper.

```
act = m.get("act")[:, 0].clamp(min=0.0)[p.parent]
f = p.fibre
gate = self.amplitude * self._strength_t[p.parent] * act
```

```
if self.taper:
    gate = gate * torch.sin(math.pi * p.s.clamp(0, 1)).clamp(min=0.0) ** 0.5
```

The full tensor equation

$$\boldsymbol{\sigma}_p^{\mathrm{act}} = A\, w_m\, a_m\, T(s_p)\big[1 + \beta(\lambda_p - 1)\big]_+ \mathbf{f}_p \mathbf{f}_p^{\mathsf{T}} \tag{47}$$

then appears directly as a rank-one outer product. Here the optional force–length factor is evaluated from the deformation gradient.

Listing 5: `prototype/eye/muscle_ops.py`: fibre stretch, force–length factor, and rank-one active stress.

```
if self.stretch_activation != 0.0:
    lam = torch.bmm(p.F, f[:, :, None]).squeeze(-1).norm(dim=1).clamp(min=1e-6)
    gate = gate * (1.0 + self.stretch_activation * (lam - 1.0)).clamp(min=0.0)
if mask is not None:
    gate = gate * mask.float()
p.active_stress = gate[:, None, None] * (f[:, :, None] * f[:, None, :])
```

For Eye G, `stretch_activation=0`, so the two force–length lines are inactive and the implemented tensor is exactly the Eye G equation with the square-root sine taper.

A.3 Building the passive elastic stress

The passive fixed-corotated stress used in the lecture note is

$$\boldsymbol{\sigma}^{\mathrm{el}} = 2\mu(F - R)F^{\mathsf{T}} + \lambda J(J - 1)I. \tag{48}$$

The implementation constructs the proper polar rotation R from the deformation gradient, computes $J = \det(F)$, and then evaluates the stress tensor.

Listing 6: `prototype/eye/muscle_ops.py`: 3-D polar rotation and fixed-corotated stress for the muscle body.

```
F, C, mass = p.F, p.C, p.mass
eye = torch.eye(D, device=dev).expand(p.n, D, D)
J = torch.linalg.det(F)
U, S, Vh = torch.linalg.svd(F)
U = U.clone(); Vh = Vh.clone()
U[torch.det(U) < 0, :, -1] *= -1
Vh[torch.det(Vh) < 0, -1, :] *= -1
R = U @ Vh
stress = 2 * p.mu[:, None, None] * ((F - R) @ F.transpose(-2, -1)) \
         + eye * (p.la * J * (J - 1))[:, None, None]
```

The stock particle scatter implements the same constitutive equation. The muscle-specific implementation differs only in where it reads the active stress.

A.4 Adding the active stress to the material stress

The total stress is

$$\boldsymbol{\sigma}_p = \boldsymbol{\sigma}_p^{\mathrm{el}} + \boldsymbol{\sigma}_p^{\mathrm{act}}. \tag{49}$$

The exact implementation is the addition below.

Listing 7: `prototype/eye/muscle_ops.py`: active stress is added before momentum scatter.

```
stress = 2 * p.mu[:, None, None] * ((F - R) @ F.transpose(-2, -1)) \\
         + eye * (p.la * J * (J - 1))[:, None, None]
act = getattr(p, "active_stress", None)
if act is not None:
    stress = stress + act
```

This is the precise implementation of the conceptual statement that contraction enters the same mechanical stress balance as passive elasticity rather than appearing as an externally prescribed force on the eye.

A.5 Deformation-gradient update

The kinematic update used by the decomposed MLS-MPM implementation is

$$F^{n+1} = (I + \Delta t\, C)F^n. \tag{50}$$

The operator `mpm_strain` writes this update directly.

Listing 8: `src/plexus/operators/mpm_strain.py`: deformation-gradient update.

```
dt = sub_dt(H, self.dt_sub)
D = p.F.shape[-1]
eye = torch.eye(D, device=dev).expand(p.n, D, D)
F = (eye + dt * p.C) @ p.F
```

This matters because the active force–length law, when enabled, reads the current fibre stretch from this evolving F. The same F is also the state from which passive elastic stress is recomputed.

A.6 From stress to the affine particle-to-grid momentum matrix

The MLS-MPM affine transfer uses

$$A_p = m_p C_p - \frac{4\Delta t_{\text{sub}}}{\Delta x^2} V_p^0 \boldsymbol{\tau}_p, \tag{51}$$

where the implementation variable called `stress` is the Kirchhoff-form stress used by the scatter kernel. The active contribution therefore has the form

$$A_p^{\text{act}} = -\frac{4\Delta t_{\text{sub}}}{\Delta x^2} V_p^0 \boldsymbol{\sigma}_p^{\text{act}} \tag{52}$$

in the simplified notation of the lecture derivation.

Listing 9: `prototype/eye/muscle_ops.py`: stress-to-affine conversion and particle momentum.

```
stress = (-dt * 4 * inv_dx * inv_dx) * p.p_vol[:, None, None] * stress
affine = stress + mass[:, None, None] * C
fx, weight, flat = bspline(X, inv_dx, offsets, g.shape, periodic)
dpos_phys = (offsets[None] - fx[:, None, :]) * dx
mom = mass[:, None, None] * V[:, None, :] \\
      + (affine[:, None] @ dpos_phys[..., None]).squeeze(-1)
```

The same logic is used by the stock `mpm_scatter`; the Eye G muscle implementation changes the destination from replacement to accumulation so that more than one particle body can contribute to the same grid.

A.7 Quadratic B-spline support: how a particle talks to several grid nodes

The background field is local because each particle contributes only to its 3^D-node stencil. The kernel is the quadratic B-spline. The implementation first finds the base grid cell and then constructs the three one-dimensional weights.

Listing 10: `src/plexus/operators/mpm_grid.py`: quadratic B-spline weights and 3^D stencil.

```
base = (X * inv_dx - 0.5).floor().long()
fx = X * inv_dx - base.float()
w = torch.stack([0.5 * (1.5 - fx) ** 2,
                 0.75 - (fx - 1) ** 2,
                 0.5 * (fx - 0.5) ** 2], dim=1)
weight = torch.ones(X.shape[0], offsets.shape[0], device=X.device)
for k in range(D):
    weight = weight * w[:, oidx[:, k], k]
```

This local interpolation is what makes an entity a continuum-like body at the numerical level: neighboring particles deposit overlapping contributions to the same grid nodes instead of evolving as isolated point masses.

A.8 Accumulating two biological entities on one shared grid

The key coupling step for the eye is not a special `muscle-to-eye'' force. The muscle scatter is registered as a second implementation named` accumulate and adds its mass and momentum to the grid already populated by the globe.

Listing 11: `prototype/eye/muscle_ops.py`: multi-body particle-to-grid accumulation on the shared MPM field.

```
gm = torch.zeros(g.n_cells, device=dev)
gmv = torch.zeros(g.n_cells, D, device=dev)
gm.index_add_(0, flat, (weight * mass[:, None]).reshape(-1))
gmv.index_add_(0, flat, (weight[..., None] * mom).reshape(-1, D))
g.m = g.m + gm
g.mv = g.mv + gmv
```

The lines above are the implementation of the bottom-up entity coupling described in the lecture note. Globe particles and muscle particles remain different Plexus sets, but they contribute to the same field. The grid is therefore the intermediate mechanical representation through which the bodies exchange momentum.

A.9 Grid momentum becomes grid velocity

After all participating particle sets have scattered, the grid update computes velocity from momentum divided by mass:

$$\mathbf{v}_i = \frac{(m\mathbf{v})_i}{m_i}. \tag{53}$$

The central line is:

Listing 12: `src/plexus/operators/mpm_grid_update.py`: grid momentum-to-velocity step.

```
gm, gmv, gc = g.m, g.mv, g.c
gv = gmv / gm.clamp(min=1e-10)[:, None]
```

The grid operator then applies boundary conditions and stores `g.v` for the gather stage.

A.10 Grid-to-particle gather and advection

The grid velocity returns to the material particles using the same B-spline support. The implementation computes a weighted sum of grid velocities and then advects the particle position:

$$\mathbf{x}_p^{n+1} = \mathbf{x}_p^n + \Delta t\, \mathbf{v}_p^{n+1}. \tag{54}$$

Listing 13: `src/plexus/operators/mpm_gather.py`: weighted grid-to-particle velocity and advection.

```
gvn = g.v[flat].view(p.n, S, D)
new_V = (weight[..., None] * gvn).sum(1)
dpos_grid = offsets[None] - fx[:, None, :]
new_C = 4 * inv_dx * (weight[..., None, None] * \\
         (gvn[..., :, None] @ dpos_grid[..., None, :])).sum(1)
Xn = torch.nan_to_num(X + dt * new_V, nan=0.5)
```

This completes the material-grid-material loop. The entity has not been converted into an anonymous cloud: its particles retain their parent identity, material parameters, fibre directions, and deformation state while the grid provides the transient momentum mediator.

A.11 Proximal endpoint: the bone anchor

The proximal endpoint law in the lecture note is

$$\mathbf{a} = k(\mathbf{x}^{\text{rest}} - \mathbf{x}) - c\mathbf{v}. \tag{55}$$

The `bone_anchor` operator implements this exactly on the tagged origin cap.

Listing 14: `prototype/eye/muscle_ops.py`: proximal endpoint restoring force and damping.

```
sel = getattr(p, self.flag, None)
s = sel.float()[:, None]
acc = (self.k * (p.rest - p.get("pos")) - self.c * p.get("vel")) * s
if mask is not None:
    acc = acc * mask[:, None].float()
return {self.at: acc}
```

Because this operator emits an acceleration into the particle set, the anchor does not prescribe a new position directly. It supplies the reaction required to resist the muscle's active contraction.

A.12 Distal endpoint: the insertion is created geometrically

The distal attachment is not a separate point-force equation. The scanned geometry or analytic morphology places the tendon cap inside the globe, while the muscle belly is held at a standoff. The `blend_muscles` seed stores the two endpoint masks that later operators use:

Listing 15: `prototype/eye/blend_mpm_ops.py`: tendon and proximal-cap labels established during seeding.

```
p.register_buffer("anchored", torch.as_tensor(sarr < self.cap, device=dev))
p.register_buffer("tendon", torch.as_tensor(sarr > 1.0 - self.cap, device=dev))
p.register_buffer("active_stress", torch.zeros(p.n, 3, 3, device=dev))
```

In the scanned Eye G geometry, the seed also explicitly records the standoff and embed parameters. This is what separates desired insertion coupling from unwanted belly overlap with the sclera. The code comments describe the intended consequence: the tendon cap is the region where shared-grid coupling should act strongly, while the belly must remain clear.

A.13 How stiffness enters the code

The lecture note uses the constitutive relationship

$$\mu = \frac{E}{2(1+u)}, \qquad \lambda = \frac{Eu}{(1+u)(1-2u)}. \tag{56}$$

For the scanned muscle seed, these parameters are written directly from the specified Young's modulus and Poisson ratio:

Listing 16: `prototype/eye/blend_mpm_ops.py`: Young's modulus to Lamé parameters; particle volume and mass.

```
mu = self.youngs / (2 * (1 + self.nu))
la = self.youngs * self.nu / ((1 + self.nu) * (1 - 2 * self.nu))
p.mu = torch.full((p.n,), mu, device=dev)
p.la = torch.full((p.n,), la, device=dev)
p.p_vol = torch.as_tensor(pvol, dtype=torch.float32, device=dev)
p.mass = p.p_vol * self.density
```

This is why the ratio A/E is a meaningful first control parameter. `A` sets the magnitude of the contractile stress, while `E` sets the passive resistance encoded in the Lamé parameters. The resulting shortening is therefore an equilibrium property of the coupled active-plus-elastic material, not a distance prescribed by `muscle_contract`.

A.14 Measuring shortening rather than prescribing it

The muscle length used by the eye model is read from the material points themselves. The centreline is reconstructed by binning points along the fibre coordinate and summing the segment lengths:

Listing 17: `prototype/eye/muscle_ops.py`: current muscle length is an aggregate readout from material-point positions.

```
acc = torch.zeros(M * B, 3, device=dev).index_add_(0, key, X)
cnt = torch.zeros(M * B, device=dev).index_add_(0, key, torch.ones_like(p.s))
cen = (acc / cnt.clamp(min=1.0)[:, None]).view(M, B, 3)
seg = (cen[:, 1:] - cen[:, :-1]).norm(dim=2)
length = seg.sum(1)
new[:, la:lb] = length[:, None]
```

This is an important conceptual closure: `muscle_contract` does not contain an equation saying "shorten the muscle by q percent. It produces stress. `muscle_geometry` then measures whatever length the mechanical system produced.

A.15 Optional transverse sleeve: preventing buckling without preventing shortening

The prototype also contains a useful demonstration of how an additional mechanical constraint can change deformation without changing the basic contractile mechanism. The sleeve penalises only displacement perpendicular to the local fibre, leaving shortening along the fibre free:

$$\mathbf{d}_\perp = \mathbf{d} - (\mathbf{d}\cdot\mathbf{f})\mathbf{f}, \qquad \mathbf{a}_\perp = -k\mathbf{d}_\perp - c\mathbf{v}_\perp. \tag{57}$$

Listing 18: `prototype/eye/muscle_ops.py`: transverse sleeve constraint; axial shortening is left unpenalised.

```
f = p.fibre
d = p.get("pos") - p.rest
v = p.get("vel")
d_perp = d - (d * f).sum(1, keepdim=True) * f
v_perp = v - (v * f).sum(1, keepdim=True) * f
w = 1.0 - ((p.s - self.free_from) / \\
    max(self.free_to - self.free_from, 1e-6)).clamp(0.0, 1.0)
acc = (-self.k * d_perp - self.c * v_perp) * w[:, None]
```

This operator is a useful teaching example because it shows that stiffness" is not one scalar concept at the whole-organ level. Axial stiffness controls resistance to shortening; transverse stiffness controls path stability and buckling. The current prototype deliberately separates those roles.

A.16 One complete code-level causal chain

The complete implementation can therefore be read as the following sequence:

`muscle_contract` → `mpm_scatter[accumulate]` → `mpm_grid_update` → `mpm_gather` → `muscle_geometry / eye_pose`

with `bone_anchor` and `muscle_sleeve` supplying boundary and path constraints. The important Plexus principle is that none of these operators contains the statement rotate the eye." The rotation is a downstream consequence of local active stress, constitutive response, shared-grid momentum exchange, and anatomical attachments.

A.17 Source map sorted by implementation

The same mapping is more useful when read in the order in which the code is organized. First come the prototype-local operators that add the eye-specific constitutive and boundary behavior; then the stock Plexus MPM operators that provide the generic particle–grid mechanics. Within each implementation, the entries follow their physical role in the calculation.

Implementation	Equation / mechanism
`prototype/eye/muscle_ops.py`	
`MuscleContract`	Activation and spatial taper $T(s)$; active stress $\boldsymbol{\sigma}^{\mathrm{act}} = g\,\mathbf{f}\mathbf{f}^{\mathsf{T}}$; force–length factor when enabled.
`MPMScatterAccumulate`	Passive fixed-corotated stress; total stress; stress $\rightarrow$ affine momentum; particle $\rightarrow$ shared grid; accumulation from multiple bodies.
`BoneAnchor`	Proximal endpoint penalty force and damping.
`MuscleGeometry`	Fibre geometry, local shortening and muscle-level readouts.
`MuscleSleeve`	Transverse displacement/velocity penalty; leaves axial shortening comparatively free.
`prototype/eye/blend_mpm_ops.py`	
`BlendMuscles`	Distal tendon/insertion mask, fibre coordinate s, standoff from globe, embedding of the distal cap.
Stiffness/material setup	Young's modulus E, Poisson ratio u, Lamé parameters μ, λ, particle reference volume and mass.
`src/plexus/operators/mpm_strain.py`	
`MPMStrain`	Deformation-gradient evolution $F^{n+1} = (I + \Delta t\, C)F^n$.
`src/plexus/operators/mpm_grid.py` / `mpm_scatter.py`	
Stock scatter operators	Quadratic B-spline particle-to-grid transfer and grid-node mass/momentum accumulation.
`src/plexus/operators/mpm_grid_update.py`	
Grid update	External forces, damping, walls, and conversion from grid momentum to grid velocity.
`src/plexus/operators/mpm_gather.py`	
Grid gather	Quadratic B-spline grid-to-particle velocity interpolation and particle advection.

The source files are public in the Plexus repository. In particular, the current prototype explicitly describes `mpm_grid` as a shared background field and `mpm_scatter[accumulate]` as the mechanism by which two particle sets contribute to the same grid; the `muscle_contract` operator stores active stress on the muscle particle set rather than applying a force directly to the globe.

Code provenance for the fibre construction

The fibre-construction snippets in this appendix correspond to the public Plexus implementation inspected on 18 August 2026. In `blend_mpm_ops.py`, `path_coordinate` and `fibre_from_s` are the functions that create s_p and $\boldsymbol{f}_p$, `BlendMuscles.forward` calls them and registers the resulting `fibre` buffer, and `muscle_contract` reads that buffer during the dynamical step. See the repository pages for `prototype/eye/blend_mpm_ops.py` and `prototype/eye/muscle_ops.py`. These implementation details are the basis for the line-numbered listings above.

References

[1] Allier, C. E., *Plexus*, `prototype/eye/blend_mpm_ops.py`, current `main` branch, inspected 18 August 2026.

[2] Allier, C. E., *Plexus*, `prototype/eye/muscle_ops.py`, current `main` branch, inspected 18 August 2026.